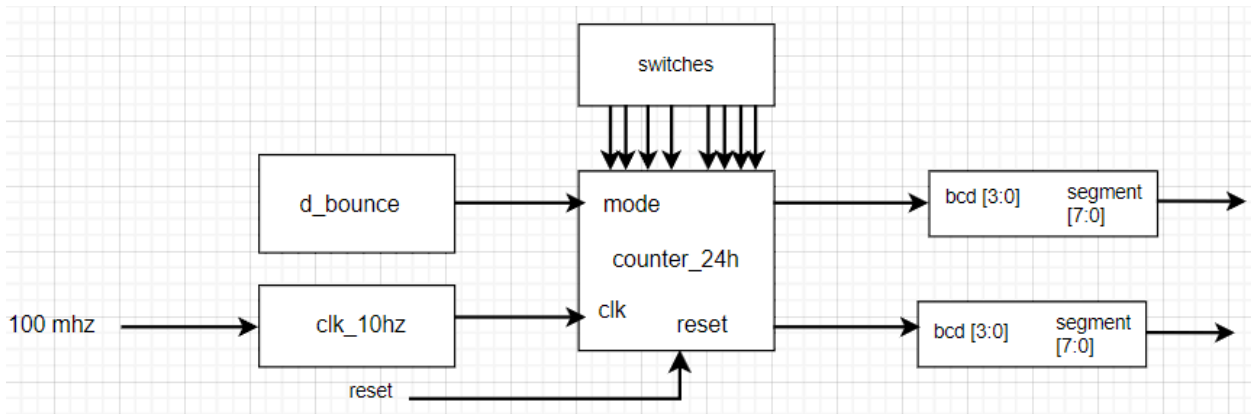


Digital Logic Design – Lab 7

- 1. Topic: Adjustable 24-hour clock
- 2. Design structure:

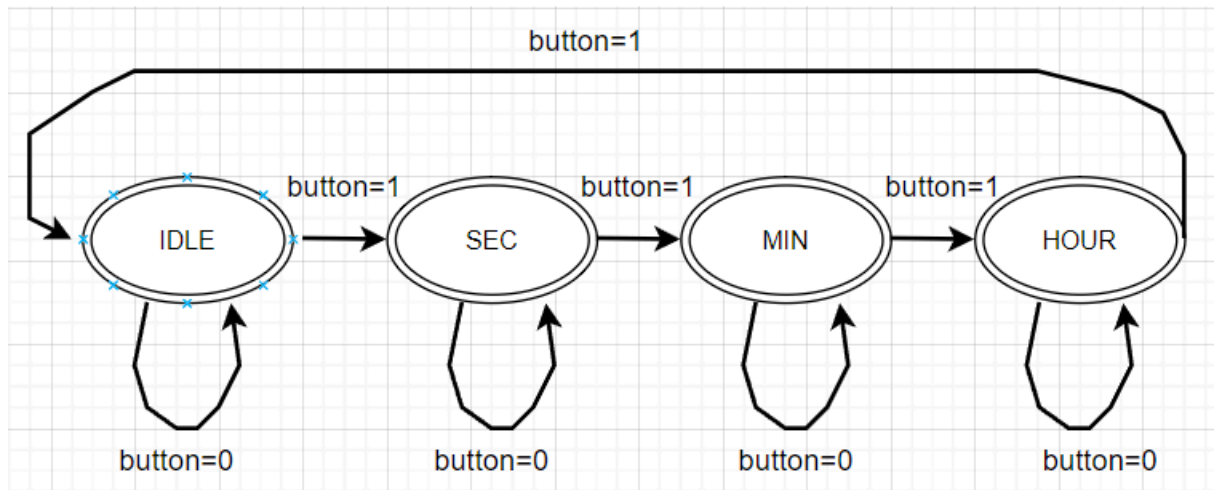
Here is the block diagram



Here is the state transition table

State	button = 0	button = 1
IDLE	IDLE	SEC
SEC	SEC	MIN
MIN	MIN	HOUR
HOUR	HOUR	IDLE

Here is the state transition graph



3. Code:

- `d_bounce.v`

```

22 // module of debounce module, with frequency of 1000 hz
23 module d_bounce(clk, rst, button, button_debounce);
24     input clk, rst, button;
25     output button_debounce;
26
27     wire [2:0] Q;
28     wire clk_1000hz_out, 1;
29
30     // generate clock
31     clk_1000hz u1(clk, rst, clk_1000hz_out);
32
33     dff d1(clk_1000hz_out, rst, button, Q[0]);
34     dff d2(clk_1000hz_out, rst, Q[0], Q[1]);
35     dff d3(clk_1000hz_out, rst, Q[1], Q[2]);
36
37     and_gate and1(Q[0], Q[1], 1);
38     and_gate and2(Q[2], 1, button_debounce);
39 endmodule
40
41 // d-flip flop module
42 module dff(clk, rst, D, Q);
43     input clk, rst, D;
44     output reg Q;
45
46
47     always @(posedge clk or posedge rst) begin
48         if (rst)
49             Q <= 1'b0;
50         else
51             Q <= D;
52     end
53 endmodule
54
55 // and gate module
56 module and_gate(x, y, z);
57     input x, y;
58     output z;
59
60
61     assign z = x & y;
62 endmodule

```

- clk_dividers.v

```

22 // module that takes a 100 mhz, and produces a 1hz clock
23 module clk_1hz(clk, rst, clk_out);
24     input clk, rst;
25     output reg clk_out;
26
27     integer counter;
28     always @(posedge clk or posedge rst) begin
29         if (rst)
30             counter <= 0;
31         else if (counter == 100000000)
32             counter <= 0;
33         else
34             counter <= counter + 1;
35     end
36
37     always @(posedge clk or posedge rst) begin
38         if (rst)
39             clk_out <= 0;
40         else if (counter <= 50000000)
41             clk_out <= 1'b1;
42         else
43             clk_out <= 1'b0;
44     end
45 endmodule
46
47
48 // module that takes a 100 mhz, and produces a 1000hz clock
49 module clk_1000hz(clk, rst, clk_out);
50     input clk, rst;
51     output reg clk_out;
52
53     integer counter;
54     always @(posedge clk or posedge rst) begin
55         if(rst)
56             counter <= 0;
57         else if(counter == 100000)
58             counter <= 0;
59         else
60             counter <= counter + 1;
61     end
62
63     always @(posedge clk or posedge rst) begin
64         if (rst)
65             clk_out <= 0;
66         else if(counter <= 50000)
67             clk_out <= 1'b1;
68         else
69             clk_out <= 1'b0;
70     end
71 endmodule

```

- counters.v

```

22 // module to up-count numbers from 0~59 for seconds and minute, and 0~23 for hours
23 module counters(clk, rst, bcd1, bcd2, bcd3, bcd4, bcd5, bcd6, state, tens, ones);
24     input clk, rst;
25     input [3:0]tens, ones;
26     input [31:0] state;
27
28     output reg [3:0] bcd1, bcd2, bcd3, bcd4, bcd5, bcd6;
29
30     integer cnt1, cnt2, cnt3, cnt4, cnt5, cnt6;
31     integer a, b;
32
33     always@(posedge clk or posedge rst)begin
34         if(rst)begin
35             a<=0;
36             b<=0;
37         end
38         else begin
39             case(ones)
40                 4'b0001: a = 1;
41                 4'b0010: a = 2;
42                 4'b0011: a = 3;
43                 4'b0100: a = 4;
44
45                 4'b0101: a = 5;
46                 4'b0110: a = 6;
47                 4'b0111: a = 7;
48                 4'b1000: a = 8;
49                 4'b1001: a = 9;
50                 4'b1010: a = 9;
51                 4'b1011: a = 9;
52                 4'b1100: a = 9;
53                 4'b1101: a = 9;
54                 4'b1110: a = 9;
55                 4'b1111: a = 9;
56                 default: a = 0;
57             endcase
58
59             case(tens)
60                 4'b0001: b = 1;
61                 4'b0010: b = 2;
62                 4'b0011: b = 3;
63                 4'b0100: b = 4;
64                 4'b0101: b = 5;
65                 4'b0110: b = 6;
66                 4'b0111: b = 7;

```

```

66 |             4'b1000: b = 8;
67 |             4'b1001: b = 9;
68 |             4'b1010: b = 9;
69 |             4'b1011: b = 9;
70 |             4'b1100: b = 9;
71 |             4'b1101: b = 9;
72 |             4'b1110: b = 9;
73 |             4'b1111: b = 9;
74 |             default: b = 0;
75 |         endcase
76 |     end
77 | end
78 |
79 | always@(posedge clk or posedge rst) begin //for counter cnt1
80 |     if(rst) begin
81 |         cnt1<=0;
82 |         cnt2<=0;
83 |         cnt3<=0;
84 |         cnt4<=0;
85 |         cnt5<=0;
86 |         cnt6<=0;
87 |     end
88 |     else
89 |         case(state)
90 |             0:begin
91 |                 cnt1<=cnt1+1;
92 |                 if(cnt1>=9)begin
93 |                     cnt1<=0;
94 |                     cnt2<=cnt2+1;
95 |                     if(cnt2>=5)begin
96 |                         cnt2<=0;
97 |                         cnt3<=cnt3+1;
98 |                         if(cnt3>=9)begin
99 |                             cnt3<=0;
100 |                             cnt4<=cnt4+1;
101 |                             if(cnt4>=5)begin
102 |                                 cnt4<=0;
103 |                                 cnt5<=cnt5+1;
104 |                                 if(cnt5>=9)begin
105 |                                     cnt5<=0;
106 |                                     cnt6<=cnt6+1;
107 |                                 end
108 |                                 else if(cnt5>=3 && cnt6>=2)begin
109 |                                     cnt5<=0;

```

```

110 | cnt6<=0;
111 | end
112 | end
113 | end
114 | end
115 | end
116 | end
117 | 1:begin
118 |     cnt1<=a;
119 |     cnt2<=b;
120 |     if(a > 9)begin
121 |         cnt1<=9;
122 |     end
123 |     if(b > 5)begin
124 |         cnt1<=9;
125 |         cnt2<=5;
126 |     end
127 | end
128 | 2:begin
129 |     cnt3<=a;
130 |     cnt4<=b;
131 |     if(a > 9)begin
132 |         cnt3<=9;
133 |     end
134 |     if(b > 5)begin
135 |         cnt3 <=9;
136 |         cnt4<=5;
137 |     end
138 | end
139 | 3:begin
140 |     cnt5<=a;
141 |     cnt6<=b;
142 |     if(b < 2 & a > 9)begin
143 |         cnt5<=9;
144 |     end
145 |     else if(b >= 2 & a >= 3)begin
146 |         cnt5<=3;
147 |     end
148 |     if(b > 2)begin
149 |         cnt5<=3;
150 |         cnt6<=2;
151 |     end
152 | end
153 | endcase

```

```

154 | end
155 |
156 | always @(cnt1) begin
157 |     case(cnt1)
158 |         1      : bcd1 = {4'b0001};
159 |         2      : bcd1 = {4'b0010};
160 |         3      : bcd1 = {4'b0011};
161 |         4      : bcd1 = {4'b0100};
162 |         5      : bcd1 = {4'b0101};
163 |         6      : bcd1 = {4'b0110};
164 |         7      : bcd1 = {4'b0111};
165 |         8      : bcd1 = {4'b1000};
166 |         9      : bcd1 = {4'b1001};
167 |         default: bcd1 = {4'b0000};
168 |     endcase
169 | end
170 |
171 | always @(cnt2) begin
172 |     case(cnt2)
173 |         1      : bcd2 = {32'b0001};
174 |         2      : bcd2 = {32'b0010};
175 |         3      : bcd2 = {32'b0011};
176 |         4      : bcd2 = {32'b0100};
177 |         5      : bcd2 = {32'b0101};
178 |         default: bcd2 = {32'b0000};
179 |     endcase
180 | end
181 |
182 | always @(cnt3) begin
183 |     case(cnt3)
184 |         1      : bcd3 = {4'b0001};
185 |         2      : bcd3 = {4'b0010};
186 |         3      : bcd3 = {4'b0011};
187 |         4      : bcd3 = {4'b0100};
188 |         5      : bcd3 = {4'b0101};
189 |         6      : bcd3 = {4'b0110};
190 |         7      : bcd3 = {4'b0111};
191 |         8      : bcd3 = {4'b1000};
192 |         9      : bcd3 = {4'b1001};
193 |         default: bcd3 = {4'b0000};
194 |     endcase
195 | end
196 |
197 | always @(cnt4) begin

```

```

198     case(cnt4)
199         1      : bcd4 = {4'b0001};
200         2      : bcd4 = {4'b0010};
201         3      : bcd4 = {4'b0011};
202         4      : bcd4 = {4'b0100};
203         5      : bcd4 = {4'b0101};
204         default: bcd4 = {4'b0000};
205     endcase
206 end
207
208 always @(cnt5) begin
209     case(cnt5)
210         1      : bcd5 = {4'b0001};
211         2      : bcd5 = {4'b0010};
212         3      : bcd5 = {4'b0011};
213         4      : bcd5 = {4'b0100};
214         5      : bcd5 = {4'b0101};
215         6      : bcd5 = {4'b0110};
216         7      : bcd5 = {4'b0111};
217         8      : bcd5 = {4'b1000};
218         9      : bcd5 = {4'b1001};
219         default: bcd5 = {4'b0000};
220     endcase
221 end
222
223 always @(cnt6) begin
224     case(cnt6)
225         1      : bcd6 = {4'b0001};
226         2      : bcd6 = {4'b0010};
227         default: bcd6 = {4'b0000};
228     endcase
229 end
230 endmodule
231

```

- **bcd_to_7seg.v**

```

22 // module to display bcd number to corresponding LEDs segment
23 module bcd_to_7seg(bcd, segment);
24     input [3:0] bcd;
25     output reg [7:0] segment;
26
27     always @(bcd) begin
28         case({bcd})
29             4'b0001 : segment = {8'b01100000};
30             4'b0010 : segment = {8'b11011010};
31             4'b0011 : segment = {8'b11110010};
32             4'b0100 : segment = {8'b01100110};
33             4'b0101 : segment = {8'b10110110};
34             4'b0110 : segment = {8'b10111110};
35             4'b0111 : segment = {8'b11100000};
36             4'b1000 : segment = {8'b11111110};
37             4'b1001 : segment = {8'b11110110};
38             4'b1010 : segment = {8'b11101110};
39             4'b1011 : segment = {8'b00111110};
40             4'b1100 : segment = {8'b10011100};
41             4'b1101 : segment = {8'b01111010};
42
43             4'b1110 : segment = {8'b10011110};
44             4'b1111 : segment = {8'b10001110};
45             default: segment = {8'b11111100};
46         endcase
47     end
48 endmodule

```

- clock_24_set.v

```

22 // module that takes all other modules to produce a 24 hour clock that you can set
23 // second, minute, and hour. You can switch modes by pressing a button, and then
24 // setting the tens and ones of the clock by changing switches
25 module clock_24h_set(clk, button, rst, left_segment, right_segment, position, tens, ones);
26     input clk, rst, button;
27     input [3:0] tens, ones;
28
29     output [7:0] left_segment, right_segment;
30     output reg [5:0] position;
31
32     // generate different frequency clocks
33     wire clk_1000hz_out, clk_1hz_out, button_debounced;
34
35     d_bounce(clk, rst, button, button_debounced);
36     clk_1000hz(clk, rst, clk_1000hz_out);
37     clk_1hz(clk, rst, clk_1hz_out);
38
39     // intermediate variables
40     wire [3:0] bcd1, bcd2, bcd3, bcd4, bcd5, bcd6;
41     integer x, a, b;
42     reg [3:0] bcd_out;
43
44     integer state;
45     integer idle = 0;
46     integer sec = 1;
47     integer min = 2;
48     integer hr = 3;
49
50     always @(posedge clk_1hz_out or posedge rst) begin
51         if(rst)
52             state<=idle;
53         else begin
54             case(state)
55                 idle:begin
56                     if(button_debounced)
57                         state<=sec;
58                     else
59                         state<=idle;
60                 end
61                 sec:begin
62                     if(button_debounced)
63                         state<=min;
64                     else
65                         state<=sec;

```

```

66 |         end
67 |         min:begin
68 |             if(button_debounced)
69 |                 state<=hr;
70 |             else
71 |                 state<=min;
72 |             end
73 |         hr:begin
74 |             if(button_debounced)
75 |                 state<=idle;
76 |             else
77 |                 state<=hr;
78 |             end
79 |         endcase
80 |     end
81 | end
82 |
83 | counters(clk_1hz_out, rst, bcd1, bcd2, bcd3, bcd4, bcd5, bcd6, state, tens, ones);
84 |
85 | always @(posedge clk_1000hz_out or posedge rst)begin
86 |     if(rst)
87 |         x<=0;

```

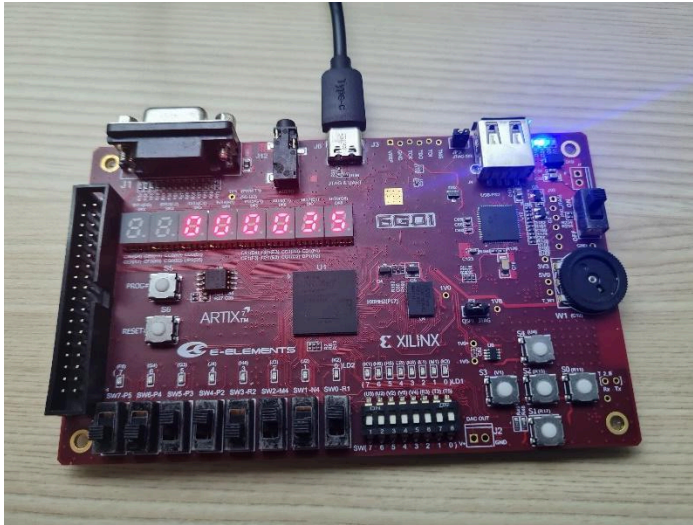
```

88     else if (x==5)
89         x<=0;
90     else
91         x<=x+1;
92     end
93
94     always @(posedge clk_1000hz_out)begin
95         case(x)
96             0:begin
97                 bcd_out<=bcd1;
98                 position<=6'b000001;
99             end
100            1:begin
101                bcd_out<=bcd2;
102                position<=6'b000010;
103            end
104            2:begin
105                bcd_out<=bcd3;
106                position<=6'b000100;
107            end
108            3:begin
109                bcd_out<=bcd4;
110
111                position<=6'b001000;
112            end
113            4:begin
114                bcd_out<=bcd5;
115                position<=6'b010000;
116            end
117            5:begin
118                bcd_out<=bcd6;
119                position<=6'b100000;
120            end
121        endcase
122    end
123
124    // display LEDs
125    bcd_to_7seg seg_left(bcd_out, left_segment);
126    bcd_to_7seg seg_right(bcd_out, right_segment);
127 endmodule

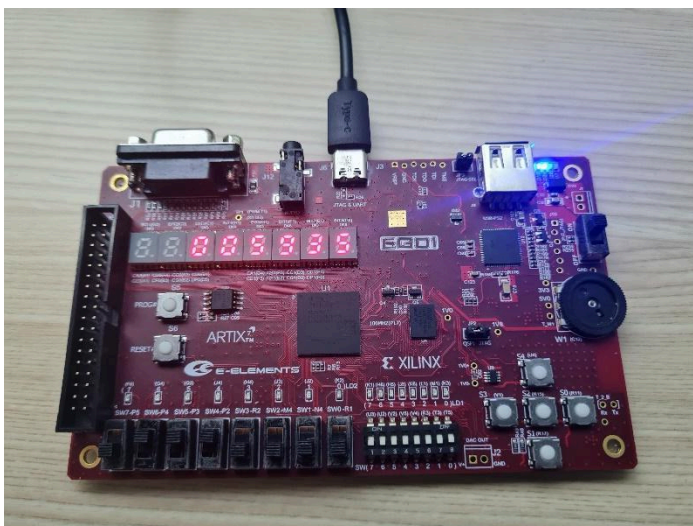
```

4. Result: the demo of the project is in the following video

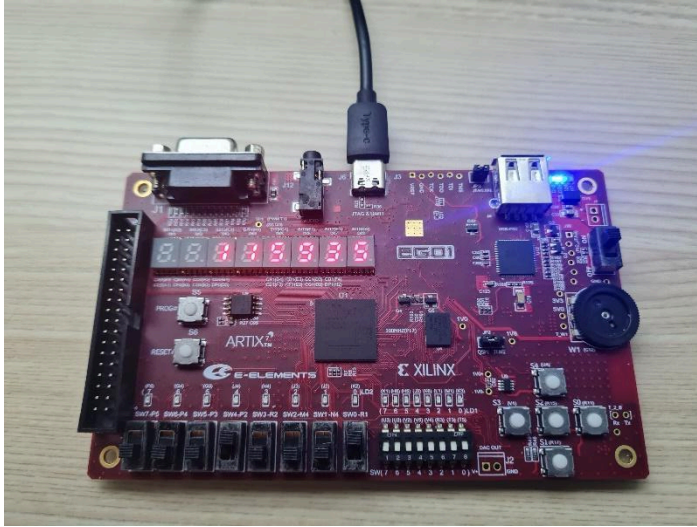
<https://www.youtube.com/watch?v=1IU0Sz6sn7g>. We designed an adjustable clock, once you press R11 button, you will set the seconds, minutes and hours section after every press of R11. To adjust the numbers you use switches, the 4 leftmost switches control the tens, and the 4 rightmost switches control the ones. If the switches are set to display a number outside the max number for that given section (hours maximum is 23, and minutes and second maximum is 59), the maximum number is displayed. Example: if you input for hour section 90, the clock displays 23. After the setting is done, you just need to press R11 again to continue the up-counting of the clock



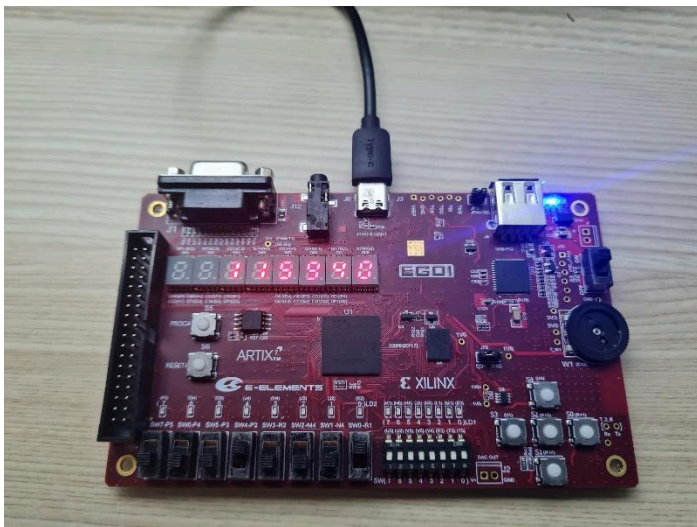
After pressing R11, you are able to change the seconds section. As seen, the left switches are 0011 = 3, and right switches are 0101 = 5, so 35 is displayed in the clock



Pressing R11 again, you will change the minutes section. As seen, the left switches are 0111 = 7, and right switches are 0000 = 0, but 70 is a number that surpasses the maximum of minutes section, so 59 is displayed instead



Press R11 again, and you will change the hour section. As seen, the left switches are 0001 = 1, and right switches are 0001 = 1, so 11 is displayed in the clock



After pressing R11 again, you can see the clock resumes the up-count

5. Reflection

In this lab, we learned how to adjust the clock based on Finite State Machine (FSM), to decide what to do for a next state scenario. We also needed to work with clocks at different frequencies to controls button debouncing, the display of LEDs one by one at high frequencies to give illusion of 6 LEDs being ON at the same time, and the frequency at which the counter goes up